

Today 11:02 AM

Lab_4_VibeCoding_Student_handout.pdf

See this and tell what i have to do i have assigned one github repo link

https://github.com/SETAPESU26/27_centipede

this is the repo link please check this and tell what i need to do please

Fetches: GitHub - SETAPESU26/27_centipede: Implementation of Centipede in Pygame

I read both the handout and your repo. Your game is Centipede, built in Python with Pygame. It has one bug and three small features to add.

Step by step

1. Set up

Fork the repo to your personal GitHub account (or clone it and push to a new repo of your own). Do not raise a PR to SETAPESU26.

Install Python 3.10+ and run `pip install pygame`.

Run `python game.py`. Controls: arrow keys to move, Space to fire, R to reset.

2. Record the "before" video

Record about 10 seconds of gameplay. Shoot a mushroom to show the bug: it disappears after 3 hits instead of 4.

3. Do the 4 tasks, one at a time, with an AI tool

Use Claude, ChatGPT or Gemini. Aim to get each task done in 3 to 4 prompts, and make a separate git commit for each task.

Task What to do

1. Fix bug In `hit_mushroom`, the comparison against `MUSHROOM_HP` (which is 4) is wrong, so mushrooms die after 3 hits. Fix it so they survive 4.

2. `mushroom_color(hp)` Return an (r, g, b) color, or None for the

default. Idea: flash white on the frame the mushroom is hit.

3. `on_segment_hit(segment, score)` Called when a body segment is destroyed. Idea: a spark effect at the segment's position, or a head-shot bonus.

4. `wave_speed_bonus(wave)` Return a speed multiplier, e.g. $1 + 0.15 * (\text{wave} - 1)$, so later waves are faster.

A good first prompt for the AI is: paste your whole `game.py`, paste the Task 1 description, and ask it to find the bug and give the minimal fix. Then do the same for each following task.

4. Record the "after" video

Record about 10 seconds that show the mushroom needing 4 hits, the new color flash, the spark or bonus effect, and the speed increase in later waves.

5. Test against Expected Behavior

Check that the centipede drops a row at borders and mushrooms, a head shot shortens the chain, a body shot splits it into two chains, touching a segment costs a life, and clearing all segments starts a faster wave.

6. Push to your repo with a Lab-4 folder

The handout asks you to upload these:

Before and after videos

Updated `game.py`

Chat history exported as a PDF or doc

The repo README says the submission is the two videos and a link to your full chat. Since the two differ slightly, I'd include everything (video, code, chat PDF and chat link) so you're covered. Ask your instructor if you're unsure.

Suggested commits

```
git commit -m "Task 1: fix mushroom durability bug"
```

```
git commit -m "Task 2: implement mushroom_color"
```

```
git commit -m "Task 3: implement on_segment_hit"
```

```
git commit -m "Task 4: implement wave_speed_bonus"
```

This is an individual assignment, so write your own prompts and keep the whole chat history, since it is part of what you submit.

If you paste the contents of game.py here, I can walk you through the bug and each feature with example prompts. If you do that in this chat, you can export it as your chat history.

Claude is AI and can make mistakes. Please double-check responses.

Claude finished the response

check this and tell me

Yep bro, I checked what you pasted. **The Claude response is basically correct**, but I'd make the workflow a little clearer so you don't accidentally miss something in the lab submission.

What you actually have to do

Your assigned repo is:

[GitHub – 27_centipede ↗](#)

It's a **Pygame Centipede game**, and your Lab 4 is basically an **AI-assisted coding task**.

You have **4 coding tasks**:

Task	What you need to do
1	Fix the mushroom bug — it currently dies after 3 hits , but should take 4 hits
2	Implement <code>mushroom_color(hp)</code> — change mushroom appearance based on HP, e.g. flash white when hit
3	Implement <code>on_segment_hit(segment, score)</code> — add an effect when a centipede segment is destroyed, e.g. spark
4	Implement <code>wave_speed_bonus(wave)</code> — make later waves progressively faster

Important: Don't just change everything at once

The assignment is specifically about **Vibe Coding / AI-assisted development**, so you should:

Task 1 → AI prompt → modify code → test → Git commit

then

Task 2 → AI prompt → modify code → test → Git commit

and so on.

So you should have **4 separate commits**.

For example:

</> Plain text

Task 1: Fix mushroom durability bug
Task 2: Implement mushroom color feedback
Task 3: Implement segment hit effect
Task 4: Implement wave speed bonus

Before you start coding

You should first get the repo onto **your own GitHub account**.

Don't work directly on the SETAPESU26/27_centipede repository.

Ideally:

</> Plain text

SETAPESU26/27_centipede
↓
Fork
↓
Your GitHub account
↓
Your copy of the project

Then clone **your fork** to your computer.

Then record BEFORE video

Before changing anything, run the game and record roughly **10 seconds**.

You need to demonstrate the original behavior/bug.

For example:

1. Start the game.

2. Shoot a mushroom.
3. Show that it disappears after **3 hits**.
4. Stop recording.

Keep this as something like:

`</>` Plain text

before.mp4

Then do the 4 tasks

For each task, use ChatGPT/Claude/Gemini.

Very important: keep your AI conversation/history because the assignment apparently asks for the **chat history** as part of the submission.

You can prompt something like:

I am working on Lab 4 Vibe Coding. Here is my current `game.py` and the Task 1 requirement. Please identify the bug and suggest the minimum code change needed. Do not modify unrelated parts of the game.

Then paste your actual `game.py` .

After Task 1 works:

`</>` Bash

```
git add .
git commit -m "Task 1: Fix mushroom durability bug"
git push
```

Then move to Task 2.

After all 4 tasks

Your game should demonstrate:

- ☒ Mushroom requires **4 hits**
- ☒ Mushroom has a **visual color/flash effect** when hit
- ☒ Destroying a centipede segment produces a **hit effect/bonus**

✓ Later waves become **faster**

Then record your **after video** showing these changes.

Something like:

⌕ Plain text

```
before.mp4
after.mp4
```

You also need to test the original expected behavior

Don't only test the four new features.

Check that:

- Centipede moves correctly
- It drops down at the borders
- Mushrooms affect its movement
- Shooting the **head** shortens the chain
- Shooting a **body segment** splits the chain
- Touching a segment costs a life
- Destroying all segments starts a new wave
- New waves are faster

Submission folder

I'd make your repo look something like:

⌕ Plain text

```
27_centipede/
|
├─ game.py
├─ README.md
├─ ...
|
└─ Lab-4/
    ├─ before.mp4
    ├─ after.mp4
    └─ chat_history.pdf
```

If your handout specifically gives a different folder/file naming format, **follow the handout over this example.**

So your complete workflow is:

1. Fork repo

↓

2. Clone your fork

↓

3. Run original game

↓

4. Record BEFORE video

↓

5. Task 1 → AI → code → test → commit

↓

6. Task 2 → AI → code → test → commit

↓

7. Task 3 → AI → code → test → commit

↓

8. Task 4 → AI → code → test → commit

↓

9. Test entire game

↓

10. Record AFTER video

↓

11. Export AI chat history to PDF

↓

12. Put required files in Lab-4

↓

13. Push everything to your GitHub repo

↓

14. Submit your GitHub link + whatever the handout asks for

One thing though: I haven't actually been given the contents of

Lab_4_VibeCoding_Student_handout.pdf in this chat, so I can't verify the Claude response against the handout itself. If you **upload the PDF here**, I can go through the handout and tell you **exactly what to do from Step 1, including what screenshots/videos/files you need to submit**, without guessing.



Lab_4_VibeCoding...
PDF

I have assigned one github repo link

https://github.com/SETAPESU26/27_centipede

Tell what i need to do 1st of all i forked that repo link tell now

Yes bro 👍 Since **you already forked the assigned repo**, you're at the correct starting point.

I checked the actual Lab 4 handout. The first instructions are: go to the assigned repo, read its README, then clone/fork it, run the Python game, and **record 10 seconds before making any changes**.  Lab_4_VibeCoding_Student_handout

● DON'T CHANGE ANY CODE YET

Do these steps in this exact order.

STEP 1 — You already did this ✓

You forked:

SETAPESU26/27_centipede

Now you should have **your own copy** under your GitHub account.

Go to your GitHub profile → **Repositories**.

You should see something like:

⌕ Plain text

27_centipede

Open **your fork**, not the original SETAPESU26/27_centipede .

STEP 2 — Read the README

Inside your fork, click:

README.md

Read it completely.

The handout specifically says to go through the README because **the deliverables/features are listed there**.  Lab_4_VibeCoding_Student_handout

Do NOT start coding yet.

STEP 3 — Clone YOUR fork to your computer

On your fork's GitHub page:

Click:

Code → HTTPS → Copy URL

It should look roughly like:

</> Plain text

```
https://github.com/YOUR_USERNAME/27_centipede.git
```

Then open **VS Code**.

Press:

</> Plain text

```
Ctrl + Shift + P
```

Search:

</> Plain text

```
Git: Clone
```

Paste your GitHub URL.

Choose where you want to save it.

Then:

Open the cloned repository in VS Code.

STEP 4 — Check the files

In VS Code, you should see the project files.

Look for something like:

</> Plain text

```
27_centipede
|
├─ game.py
├─ README.md
└─ ...
```

Don't modify anything yet.

STEP 5 — Install the requirements

Open VS Code terminal:

Terminal → New Terminal

First check Python:

</> Bash

```
python --version
```

Then install Pygame if required:

</> Bash

```
pip install pygame
```

STEP 6 — RUN THE ORIGINAL GAME 🎮

This is VERY important.

You need to run the **original untouched version** before making changes.

If the main file is `game.py`, run:

</> Bash

```
python game.py
```

If README gives a different command, follow the README.

The handout explicitly says to **run the Python-based code** before modifications.

 Lab_4_VibeCoding_Student_handout

STEP 7 — RECORD THE BEFORE VIDEO

This happens **before** you change anything.

Record approximately **10 seconds** of the original game.

The handout specifically requires a 10-second video before making changes.

 Lab_4_VibeCoding_Student_handout

For example:

 Plain text

```
Lab-4/  
before.mp4
```

Don't worry about making it fancy.

Just demonstrate the original game working/broken.

THEN STOP

Once you have:

- ☒ Forked repo
- ☒ Cloned YOUR fork
- ☒ Opened it in VS Code
- ☒ Installed requirements
- ☒ Run original game
- ☒ Recorded 10-second BEFORE video

Come back and send me a screenshot of your VS Code/project folder + README.

Then I'll tell you **exactly what to do next**, including:

- what the first bug is
- the exact AI prompt to use
- how to make the first change
- how to test it
- how to make the **first Git commit**
- then Task 2, Task 3, Task 4
- exactly what to put inside Lab-4

- exactly what to submit

And remember: **separate commit for every task** and **do NOT** create a PR to **SETAPESU26** . The handout explicitly requires separate commits and says not to raise a PR to the main repo.

 Lab_4_VibeCoding_Student_handout

So right now your only job is: clone your fork → run the original game → record the 10-second before video.

create a new fork or what?

No bro 🤖 if you already forked the assigned repo, **DO NOT** create another fork.

You only need **one** fork.

Your flow is:

⌄ Plain text

```
SETAPESU26/27_centipede    ← Original repo
    ↓
  YOUR FORK                ← You already created this ✅
    ↓
Clone to your PC
    ↓
Make changes
    ↓
Commit each task
    ↓
Push to YOUR fork
```

Right now, do this:

1. Go to your **GitHub** profile
2. Open your fork of 27_centipede
3. Click **Code**
4. Copy the **HTTPS URL**
5. Clone that URL in VS Code

Do not fork again.

The handout says “**Clone or Fork the repo**”, so since you've already chosen **Fork**, you're done with that step.  Lab_4_VibeCoding_Student_handout

If you send me a **screenshot of your forked GitHub page**, I'll tell you exactly what to click next.



is this only

YES brooo 🤖 ✅ THIS IS IT.

That screenshot shows:

Surya... / 27_centipede
forked from SETAPESU26/27_centipede

So you have successfully created your fork. ✅

Now DON'T click Fork again.

Your next step is:

1 Click the green Code button

You'll see options like:

HTTPS | SSH | GitHub CLI

Choose HTTPS and copy the URL.

2 Open VS Code

Press:

Ctrl + Shift + P

Search:

Git: Clone

Paste the URL you copied.

3 Open the cloned folder

Once VS Code asks whether you want to open the cloned repository → **Yes/Open**.

4 THEN we'll run the original game

Don't edit `game.py` yet.

We first need to run the untouched game and make your **10-second BEFORE video**, because the handout explicitly requires that before making changes.  Lab_4_VibeCoding_Student_handout

👉 So right now: click **Code** → **HTTPS** → **copy the URL**.

If you want, send me a screenshot after clicking **Code**, and I'll tell you exactly which one to click.

Downloading pygame-2.6.1.tar.gz (14.8 MB)

14.8/14.8 MB 15.1 MB/s

0:00:01

Installing build dependencies ... done

Getting requirements to build wheel ... error

error: subprocess-exited-with-error

× Getting requirements to build wheel did not run successfully.

| exit code: 1

↳ [112 lines of output]

Skipping Cython compilation

WARNING, No "Setup" File Exists, Running

"buildconfig/config.py"

Using WINDOWS configuration...

Making dir :prebuilt_downloads:

Downloading... <https://www.libsdl.org/release/SDL2-devel-2.28.4-VC.zip> 25ef9d201ce3fd5f976c37dddedac36bd173975c

Unzipping :prebuilt_downloads\SDL2-devel-2.28.4-VC.zip:

Downloading...

https://www.libsdl.org/projects/SDL_image/release/SDL2_image-devel-2.0.5-VC.zip 137f86474691f4e12e76e07d58d5920c8d844d5b

Unzipping :prebuilt_downloads\SDL2_image-devel-2.0.5-VC.zip:

Downloading... [https://github.com/libsdl-](https://github.com/libsdl-org/SDL_ttf/releases/download/release-2.20.1/SDL2_ttf-devel-2.20.1-VC.zip)

[org/SDL_ttf/releases/download/release-2.20.1/SDL2_ttf-devel-2.20.1-VC.zip](https://github.com/libsdl-org/SDL_ttf/releases/download/release-2.20.1/SDL2_ttf-devel-2.20.1-VC.zip) 371606aceba450384428fd2852f73d2f6290b136

Unzipping :prebuilt_downloads\SDL2_ttf-devel-2.20.1-VC.zip:

Downloading... [https://github.com/libsdl-](https://github.com/libsdl-org/SDL_mixer/releases/download/release-2.6.2/SDL2_mixer-devel-2.6.2-VC.zip)

[org/SDL_mixer/releases/download/release-2.6.2/SDL2_mixer-devel-2.6.2-VC.zip](https://github.com/libsdl-org/SDL_mixer/releases/download/release-2.6.2/SDL2_mixer-devel-2.6.2-VC.zip)

000e3ea8a50261d46dbd200fb450b93c59ed4482

Unzipping :prebuilt_downloads\SDL2_mixer-devel-2.6.2-VC.zip:

Downloading...

<https://github.com/pygame/pygame/releases/download/2.1.3.dev>

4/prebuilt-x64-pygame-2.1.4-20220319.zip

16b46596744ce9ef80e7e40fa72ddbafef1cf586

Unzipping :prebuilt_downloads\prebuilt-x64-pygame-2.1.4-20220319.zip:

copying into .\prebuilt-x64

Path for SDL: prebuilt-x64\SDL2-2.28.4

...Library directory for SDL: prebuilt-x64/SDL2-2.28.4/lib/x64

...Include directory for SDL: prebuilt-x64/SDL2-2.28.4/include

Path for FONT: prebuilt-x64\SDL2_ttf-2.20.1

...Library directory for FONT: prebuilt-x64/SDL2_ttf-2.20.1/lib/x64

...Include directory for FONT: prebuilt-x64/SDL2_ttf-2.20.1/include

Path for IMAGE: prebuilt-x64\SDL2_image-2.0.5

...Library directory for IMAGE: prebuilt-x64/SDL2_image-2.0.5/lib/x64

...Include directory for IMAGE: prebuilt-x64/SDL2_image-2.0.5/include

Path for MIXER: prebuilt-x64\SDL2_mixer-2.6.2

...Library directory for MIXER: prebuilt-x64/SDL2_mixer-2.6.2/lib/x64

...Include directory for MIXER: prebuilt-x64/SDL2_mixer-2.6.2/include

Path for PORTMIDI: prebuilt-x64

...Library directory for PORTMIDI: prebuilt-x64/lib

...Include directory for PORTMIDI: prebuilt-x64/include

DLL for SDL2: prebuilt-x64/SDL2-2.28.4/lib/x64/SDL2.dll

DLL for SDL2_ttf: prebuilt-x64/SDL2_ttf-2.20.1/lib/x64/SDL2_ttf.dll

DLL for SDL2_image: prebuilt-x64/SDL2_image-2.0.5/lib/x64/SDL2_image.dll

DLL for SDL2_mixer: prebuilt-x64/SDL2_mixer-2.6.2/lib/x64/SDL2_mixer.dll

DLL for portmidi: prebuilt-x64/lib/portmidi.dll

Path for FREETYPE: prebuilt-x64

...Library directory for FREETYPE: prebuilt-x64/lib

...Include directory for FREETYPE: prebuilt-x64/include

Path for PNG not found.

...Found include dir but no library dir in prebuilt-x64.

Path for JPEG not found.

...Found include dir but no library dir in prebuilt-x64.
DLL for freetype: prebuilt-x64/lib/freetype.dll
DLL for png: prebuilt-x64/SDL2_image-2.0.5/lib/x64/libpng16-16.dll

For help with compilation see:

<https://www.pygame.org/wiki/CompileWindows>

To contribute to pygame development see:

<https://www.pygame.org/contribute.html>

Traceback (most recent call last):

File "C:\Users\purim\AppData\Local\Temp\pip-install-ofwhgmtb\pygame_28f76c3062454e2baf8b874fcc85d1f4\buildconfig\uptools.py", line 4, in <module>
 from distutils.msvccompiler import MSVCCompiler,
 get_build_architecture
ModuleNotFoundError: No module named
'distutils.msvccompiler'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

File "C:\Users\purim\AppData\Local\Temp\pip-install-ofwhgmtb\pygame_28f76c3062454e2baf8b874fcc85d1f4\buildconfig\config_win.py", line 336, in configure
 from . importuptools
File "C:\Users\purim\AppData\Local\Temp\pip-install-ofwhgmtb\pygame_28f76c3062454e2baf8b874fcc85d1f4\buildconfig\uptools.py", line 6, in <module>
 from setuptools._distutils.msvccompiler import
 MSVCCompiler, get_build_architecture
ModuleNotFoundError: No module named
'setuptools._distutils.msvccompiler'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File "C:\Users\purim\AppData\Local\Temp\pip-install-
ofwhgmtb\pygame_28f76c3062454e2baf8b874fcc85d1f4\buildcon
fig\uptools.py", line 4, in <module>
    from distutils.msvccompiler import MSVCCompiler,
get_build_architecture
ModuleNotFoundError: No module named
'distutils.msvccompiler'
```

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File
"C:\Users\purim\AppData\Local\Programs\Python\Python314\Lib\
site-
packages\pip\_vendor\pyproject_hooks\_in_process\_in_process.py
", line 389, in <module>
    main()
    ~~~~^^
File
"C:\Users\purim\AppData\Local\Programs\Python\Python314\Lib\
site-
packages\pip\_vendor\pyproject_hooks\_in_process\_in_process.py
", line 373, in main
    json_out["return_val"] = hook(**hook_input["kwargs"])
                             ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File
"C:\Users\purim\AppData\Local\Programs\Python\Python314\Lib\
site-
packages\pip\_vendor\pyproject_hooks\_in_process\_in_process.py
", line 143, in get_requires_for_build_wheel
    return hook(config_settings)
File "C:\Users\purim\AppData\Local\Temp\pip-build-env-
sr2u488t\overlay\Lib\site-packages\setuptools\build_meta.py",
line 333, in get_requires_for_build_wheel
    return self._get_build_requires(config_settings,
requirements=[])
```

~~~~~

^^^^^^^^^^^^^^^^

File "C:\Users\purim\AppData\Local\Temp\pip-build-env-sr2u488t\overlay\Lib\site-packages\setuptools\build\_meta.py", line 301, in \_get\_build\_requires

self.run\_setup()

~~~~~^

File "C:\Users\purim\AppData\Local\Temp\pip-build-env-sr2u488t\overlay\Lib\site-packages\setuptools\build_meta.py", line 520, in run_setup

super().run_setup(setup_script=setup_script)

~~~~~  
^^^^^^^^^^^^^^^^  
^

File "C:\Users\purim\AppData\Local\Temp\pip-build-env-sr2u488t\overlay\Lib\site-packages\setuptools\build\_meta.py", line 317, in run\_setup

exec(code, locals()) # noqa: S102 # exec is intentional here

~~~~  
^^^^^^^^^^^^^^^^

File "<string>", line 432, in <module>

File "C:\Users\purim\AppData\Local\Temp\pip-install-ofwhgmtb\pygame_28f76c3062454e2baf8b874fcc85d1f4\buildconfig\config.py", line 234, in main

deps = CFG.main(**kws, auto_config=auto)

File "C:\Users\purim\AppData\Local\Temp\pip-install-ofwhgmtb\pygame_28f76c3062454e2baf8b874fcc85d1f4\buildconfig\config_win.py", line 493, in main

return setup_prebuilt_sdl2(prebuilt_dir)

File "C:\Users\purim\AppData\Local\Temp\pip-install-ofwhgmtb\pygame_28f76c3062454e2baf8b874fcc85d1f4\buildconfig\config_win.py", line 453, in setup_prebuilt_sdl2

DEPS.configure()

~~~~~^

File "C:\Users\purim\AppData\Local\Temp\pip-install-ofwhgmtb\pygame\_28f76c3062454e2baf8b874fcc85d1f4\buildconfig\config\_win.py", line 338, in configure

from buildconfig import vstools

File "C:\Users\purim\AppData\Local\Temp\pip-install-ofwhgmtb\pygame\_28f76c3062454e2baf8b874fcc85d1f4\buildconfig\vstools.py", line 6, in <module>

from setuptools.\_distutils.msvccompiler import

```
MSVCCompiler, get_build_architecture
ModuleNotFoundError: No module named
'setuptools._distutils.msvccompiler'
[end of output]
```

note: This error originates from a subprocess, and is likely not a problem with pip.

```
[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
error: subprocess-exited-with-error
```

```
× Getting requirements to build wheel did not run successfully.
| exit code: 1
└> See above for output.
```

note: This error originates from a subprocess, and is likely not a problem with pip.

Ahh, I see the exact problem, bro. **Your fork is fine. The problem is your Python version.** 🤖

Your error clearly shows:

⌕ Plain text

Python314

You are using **Python 3.14**.

The repo says Python **3.10+** and asks you to install Pygame, but your `pip` is trying to **build Pygame from source** instead of installing a compatible prebuilt package. The failure is specifically around `distutils.msvccompiler`. The repo itself only requires Python 3.10+ and `pip install pygame`.



## Easiest fix: use Python 3.13

**Don't change the game code. Don't touch GitHub. Don't reinstall your repo.**

### Step 1 — Check whether you already have another Python

In the VS Code terminal, run:

⌕ Bash

```
py -0p
```

Send me the output.

It may show something like:

</> Plain text

```
-V:3.14  
-V:3.13  
-V:3.12
```

## If you have Python 3.13

Then we can simply install Pygame using that version:

</> Bash

```
py -3.13 -m pip install pygame
```

Then run the game with:

</> Bash

```
py -3.13 game.py
```

## If you DON'T have Python 3.13

Don't worry. We'll install it, then continue.

### Don't do this yet

Don't keep trying:

</> Bash

```
pip install pygame
```

with your current Python 3.14.

And **don't start modifying** `game.py` **yet**.

First we need to get the original game running. The README says the intended setup is simply `pip install pygame` followed by `python game.py` .  [GitHub](#)

**Run this now:****</> Bash**`py -0p`

Send me the screenshot/output. I'll tell you the **exact next command**.

```
PS C:\Desktop\SE LAB\27_centipede> py -0p
-V:3.14t *
C:\Users\purim\AppData\Local\Programs\Python\Python314\python3.14t.exe
-V:3.14
C:\Users\purim\AppData\Local\Programs\Python\Python314\python.exe
-V:3.12
C:\Users\purim\AppData\Local\Programs\Python\Python312\python.exe
```

Perfect brooo 🎉 ✅ You already have **Python 3.12 installed**, so we don't need to install anything else.

Your Python versions are:

- ❌ Python 3.14 — causing the Pygame problem
- ✅ Python 3.12 — use this

**Do this now**

In the **same VS Code terminal**, run:

**</> Bash**`py -3.12 -m pip install pygame`

Wait for it to finish.

You should hopefully see something like:

**</> Plain text**`Successfully installed pygame-...`**Then run the original game**

After installation finishes:

```
</> Bash
```

```
py -3.12 game.py
```

 The Centipede game should open.

**Do not change `game.py` yet.**

---

## If the game opens successfully

Your next job is:

1. Play the **original untouched game**
2. Find/show the existing behavior/bug
3. Record the required **10-second BEFORE video** — the handout says this must be done before making changes.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)
4. Then we'll start **Task 1**

## One important thing

From now on, because your computer defaults to Python 3.14, use:

```
</> Bash
```

```
py -3.12 game.py
```

rather than:

```
</> Bash
```

```
python game.py
```

for this project.

**Run this first:**

```
</> Bash
```

```
py -3.12 -m pip install pygame
```

Send me what it says after it finishes.

```
PS C:\Desktop\SE LAB\27_centipede> py -3.12 -m pip install
pygame
Collecting pygame
  Downloading pygame-2.6.1-cp312-cp312-
win_amd64.whl.metadata (13 kB)
  Downloading pygame-2.6.1-cp312-cp312-win_amd64.whl (10.6
MB)
    _____ 10.6/10.6 MB 8.1 MB/s eta
0:00:00
Installing collected packages: pygame
Successfully installed pygame-2.6.1

[notice] A new release of pip is available: 25.0.1 -> 26.2.1
[notice] To update, run:
C:\Users\purim\AppData\Local\Programs\Python\Python312\pyth
on.exe -m pip install --upgrade pip
```

YESS brooo 🧨 **Pygame installed successfully.** ✅

That means the Python problem is fixed.

**Now do ONLY this:**

In the same terminal:

```
</> Bash
```

```
py -3.12 game.py
```

This should launch the **original Centipede game**.

### ⚠️ Important

Don't edit `game.py` yet.

The lab says you must:

**Run original game → record 10-second BEFORE video → THEN make changes.**

 Lab\_4\_VibeCoding\_Student\_handout

So:

1. Run:

</> **Bash**

py -3.12 game.py

2. Play the game for a bit.
3. Make sure it works.
4. Record about **10 seconds** of the original game.
5. Keep that video safe — we'll call it **Before video**.

Once the game opens, **send me a screenshot of the game** (or tell me it opened). Then I'll guide you to the **next exact step**. 🎮



Perfect brooo 🔥 🎮 the original game is running successfully.

So far you're completely on track:

- ☒ Forked the assigned repo
- ☒ Cloned your fork
- ☒ Python 3.12 working
- ☒ Pygame installed
- ☒ Original game.py runs
- ☒ **Do not modify code yet**

**Now your next task: BEFORE video** 🎥

The handout says to **record 10 seconds before making any changes**.

 Lab\_4\_VibeCoding\_Student\_handout

For the video, I'd suggest showing:

1. Start the game.
2. Move the player/shooter.
3. Shoot the centipede/mushrooms.
4. Capture the normal gameplay for around **10 seconds**.

You don't need to explain anything in the video. Just show the original game.

## After recording

Keep the video somewhere safe, for example:

</> **Plain text**

Lab-4

└─ before.mp4



**Don't put it into Git yet unless we decide the exact submission structure from the README.**

Once you've recorded the **10-second BEFORE video**, tell me:

**Done**

Then we'll open the README and start **Task 1 using the AI prompt**, one task at a time. The handout also requires **separate commits for each task**, so we'll do that carefully.  Lab\_4\_VibeCoding\_Student\_handout



20261008-0545-16....  
File

Check this video

Worked for 11s

Analyzed

Analyzed

Yep bro, I checked the video. 

**Your BEFORE video is good.**

-  **Length: about 11.4 seconds** — requirement is 10 seconds, so that's perfectly fine.
-  It clearly shows the **original Centipede game running**.
-  You're showing actual gameplay before making changes.
-  It is suitable as your **Before video**.

So **don't record it again**. Keep this one. 

**Now you're ready for the next step**

**Do NOT modify `game.py` yet.**

Next we need to look at the repo's **README**, because the handout specifically says the features/tasks to implement are listed there.  Lab\_4\_VibeCoding\_Student\_handout

**Send me a screenshot of the full README** (scroll down and send screenshots if it's long).

Then I'll tell you **exactly what Task 1 is and the exact prompt you should give ChatGPT**, so we can do the lab properly with the required AI chat history and separate commits. `</>`

NEXT

Perfect bro. Your **before video is done**, so now we move to **Task 1**. I checked the actual README of your assigned repo, so we can follow it exactly. 

## **TASK 1 — Fix the mushroom bug**

The README says the mushroom should survive **4 hits**, but currently disappears after **3 hits**. The bug is specifically in the `hit_mushroom` function, involving the comparison with `MUSHROOM_HP`, which is `4`.  [GitHub](#)

## But remember: this is Vibe Coding

Your lab says you must use an **LLM iteratively** and critically review the suggested code.  [GitHub](#)

So **don't ask me to directly fix it and paste the code** if you want your lab chat history to demonstrate the required process.

Instead, we're going to use **this ChatGPT conversation as your LLM chat history**.

---

## STEP 1 — Open `game.py`

In VS Code, open:

 Plain text

```
27_centipede
├── .gitignore
├── README.md
└── game.py
```

Open `game.py`.

**Don't change anything.**

---

## STEP 2 — Give me the code

Since the lab wants you to prompt an AI with the code, let's do that properly.

**In VS Code:**

Press:

**Ctrl + A**

inside `game.py`

then:

**Ctrl + C**

Come back here and paste the entire `game.py` .

Then I'll help you formulate the **first prompt for Task 1**, identify what the AI response means, and we'll make the **minimum change only**.

---

## Why we're doing it this way

Your README says:

Task 1 is to fix the mushroom durability bug.  [GitHub](#)

And the handout requires you to use prompts to fix the broken code and then add the features **one by one**, with separate commits for each task.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

So our process will be:

### Plain text

```
TASK 1
↓
Paste game.py + Task 1 prompt
↓
AI analyzes bug
↓
You review suggestion
↓
Make ONLY Task 1 change
↓
Run/test game
↓
Git commit #1
```

Then we'll do Task 2 separately, then Task 3, then Task 4.

 **Don't commit anything yet.**

Paste your entire `game.py` here now. I'll take you through Task 1 step-by-step.

```
import random
import pygame
```

```
CELL, COLS, ROWS = 20, 30, 30
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4
```

```
def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit
    points, or None for the default."""
    pass

def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles,
    sounds, or bonus points here."""
    pass

def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at
    the given wave, or None for the default speed."""
    pass

class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1

    def step(self, mushrooms):
        nxt = self.col + self.dx
        if nxt < 0 or nxt >= COLS or (self.row, nxt) in mushrooms:
            self.row += self.dy
            self.dx = -self.dx
            if self.row >= ROWS - 1:
                self.row, self.dy = ROWS - 1, -1
            elif self.row <= ZONE_TOP and self.dy < 0:
                self.dy = 1
        else:
            self.col = nxt

class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
```

```
self.reset()

def reset(self):
    self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
    self.mushrooms = {}
    for _ in range(45):
        self.mushrooms[(random.randint(1, ZONE_TOP - 2),
random.randint(0, COLS - 1))] = MUSHROOM_HP
    self.respawn()
    self.spawn_wave()

def respawn(self):
    self.x = WIDTH / 2
    self.y = HEIGHT - 50
    self.bullet = None
    self.invulnerable = 1.5

def spawn_wave(self):
    length = 10 + self.wave
    self.chains = [[Segment(0, COLS - 1 - i, -1) for i in
range(length)]]
    self.timer = 0.0

def fire(self):
    if self.bullet is None and self.state == "play":
        self.bullet = pygame.Vector2(self.x, self.y - 12)

def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1
    if self.mushrooms[cell] <= 1:
        del self.mushrooms[cell]
        self.score += 5

def split_chain(self, chain, index):
    segment = chain[index]
    self.chains.remove(chain)
    left, right = chain[:index], chain[index + 1:]
    self.chains.extend(part for part in (left, right) if part)
    self.mushrooms[(segment.row, segment.col)] =
MUSHROOM_HP
```

```

self.score += 100 if index == 0 else 10
on_segment_hit(segment, self.score)

```

```

def update_bullet(self, dt):
    if self.bullet is None:
        return
    self.bullet.y -= BULLET_SPEED * dt
    cell = (int(self.bullet.y // CELL), int(self.bullet.x // CELL))
    if self.bullet.y < 0:
        self.bullet = None
    elif cell in self.mushrooms:
        self.hit_mushroom(cell)
        self.bullet = None
    else:
        for chain in self.chains:
            for index, segment in enumerate(chain):
                if (segment.row, segment.col) == cell:
                    self.split_chain(chain, index)
                    self.bullet = None
            return

def update(self, dt, keys):
    if self.state != "play":
        return
    self.invulnerable = max(0.0, self.invulnerable - dt)
    self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) *
PLAYER_SPEED * dt
    self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) *
PLAYER_SPEED * dt
    self.x = max(10, min(WIDTH - 10, self.x))
    self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10,
self.y))
    if keys[pygame.K_SPACE]:
        self.fire()
    self.update_bullet(dt)
    self.timer += dt
    effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
    while self.timer >= effective_tick:
        self.timer -= effective_tick
        for chain in self.chains:

```

```

        for segment in chain:
            segment.step(self.mushrooms)
        player_cell = (int(self.y // CELL), int(self.x // CELL))
        if self.invulnerable <= 0 and any((s.row, s.col) == player_cell
for c in self.chains for s in c):
            self.lives -= 1
            self.respawn()
            if self.lives <= 0:
                self.state = "lose"
        if not self.chains:
            self.wave += 1
            self.spawn_wave()

def draw(self, screen):
    screen.fill((8, 8, 16))
    for (row, col), hp in self.mushrooms.items():
        color = mushroom_color(hp) or (200 - (MUSHROOM_HP -
hp) * 40, 80, 170)
        center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
        pygame.draw.circle(screen, color, center, CELL // 2 - 1)
        pygame.draw.rect(screen, (230, 230, 200), (center[0] - 3,
center[1], 6, CELL // 2 - 1))
    for chain in self.chains:
        for index, segment in enumerate(chain):
            center = (segment.col * CELL + CELL // 2, segment.row *
CELL + CELL // 2)
            pygame.draw.circle(screen, (240, 200, 60) if index == 0
else (80, 220, 90), center, CELL // 2)
    if self.bullet:
        pygame.draw.rect(screen, (255, 255, 255), (self.bullet.x - 1,
self.bullet.y - 6, 3, 10))
    if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 ==
0:
        pygame.draw.polygon(screen, (80, 200, 255), [(self.x, self.y -
10), (self.x + 10, self.y + 10), (self.x - 10, self.y + 10)])
    hud = self.font.render(f"Score {self.score} Lives {self.lives}
Wave {self.wave} R = reset", True, (240, 240, 240))
    screen.blit(hud, (10, ROWS * CELL + 6))
    if self.state == "lose":
        label = self.font.render("GAME OVER - Press R", True, (255,

```

```

255, 120))
    screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT
// 2)))

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Centipede")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
                game.reset()
        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()
    pygame.quit()

if __name__ == "__main__":
    main()

```

Perfect brooo. 🍌 Now we have the **actual game.py**, so we can do Task 1 properly.

## 🔴 TASK 1 — Fix the mushroom bug

I can already see the bug in your code:

</> Python

```

def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1
    if self.mushrooms[cell] <= 1:
        del self.mushrooms[cell]
        self.score += 5

```



Your mushroom starts with:

Python

```
MUSHROOM_HP = 4
```

So the HP goes:

Plain text

```
4 → 3 → 2 → 1 → deleted
```

That means it disappears on the **3rd hit**, because after the third hit its HP becomes **1**, and this condition is true:

Python

```
if self.mushrooms[cell] <= 1:
```

It should disappear only when HP reaches **0**.

## BUT — for your Vibe Coding lab

Don't just manually change it yet.

The handout specifically says to **write prompts** and use the **Vibe Coding** tool to fix the broken code.

 Lab\_4\_VibeCoding\_Student\_handout

So let's make this your **Task 1 AI** interaction.

**Your prompt should be:**

Copy this and send it as your AI prompt:

I am working on Task 1 of my Vibe Coding lab. In this Centipede game, MUSHROOM\_HP is set to 4, but a mushroom currently disappears after 3 hits instead of 4. Please inspect the `hit_mushroom` function, identify the bug, and suggest the minimum code change required to make the mushroom survive exactly 4 hits. Do not modify any other functionality or code.

Then the AI should identify the problem with:

Python

```
if self.mushrooms[cell] <= 1:
```

and suggest changing it to:

Python

```
if self.mushrooms[cell] <= 0:
```

Then you make **ONLY** that change.

Your function should become:

Python

```
def hit_mushroom(self, cell):  
    self.mushrooms[cell] -= 1  
    if self.mushrooms[cell] <= 0:  
        del self.mushrooms[cell]  
    self.score += 5
```

That's it. Don't touch `mushroom_color` , `on_segment_hit` , or `wave_speed_bonus` yet.

## Then TEST Task 1 🎮

Save `game.py` .

Run:

Bash

```
py -3.12 game.py
```

Shoot a mushroom repeatedly.

You want:

Plain text

```
1st hit → mushroom remains  
2nd hit → mushroom remains  
3rd hit → mushroom remains  
4th hit → mushroom disappears
```

If that works → **Task 1 is complete.**

## Then make your FIRST commit

Don't commit yet until you've tested it successfully.

After testing, in the terminal:

```
</> Bash  
git status
```

Then:

```
</> Bash  
git add game.py
```

Then:

```
</> Bash  
git commit -m "Task 1: Fix mushroom durability bug"
```

Then:

```
</> Bash  
git push
```

The handout specifically requires **separate commits for each task**, so this will be your first commit.

 Lab\_4\_VibeCoding\_Student\_handout

### Important

Don't start Task 2 after changing the code until we verify Task 1 works.

So first:

Send the prompt above to me/your AI → make the `<= 1` → `<= 0` change → run the game and test it.

Then tell me "Task 1 tested", and we'll do Task 2 ( `mushroom_color` ) separately.

I am working on Task 1 of my Vibe Coding lab. In this Centipede game, MUSHROOM\_HP is set to 4, but a mushroom currently disappears after 3 hits instead of 4. Please inspect the `hit_mushroom` function, identify the bug, and suggest the minimum code change required to make the mushroom survive exactly 4 hits. Do not modify any other functionality or code.

Yes bro, that's the **correct Task 1 prompt**. 👍

The answer should be:

## Bug found

In `hit_mushroom()` :

⌕ Python

```
if self.mushrooms[cell] <= 1:
```

The mushroom starts at **4 HP**. After each hit:

⌕ Plain text

4 → 3 → 2 → 1 → 0

But the current condition deletes it when HP reaches `1` , so it disappears after the **3rd hit**.

## Minimum fix

Change only:

⌕ Python

```
if self.mushrooms[cell] <= 1:
```

to:

⌕ Python

```
if self.mushrooms[cell] <= 0:
```

So the final function is:

**</> Python**

```
def hit_mushroom(self, cell):  
    self.mushrooms[cell] -= 1  
    if self.mushrooms[cell] <= 0:  
        del self.mushrooms[cell]  
        self.score += 5
```

**No other code needs to be changed for Task 1.**

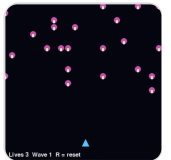
Now make that one change in `game.py`, save it, and run:

**</> Bash**

```
py -3.12 game.py
```

Test that the mushroom survives **3 shots** and disappears on the **4th shot**.

**Don't commit yet** until you've tested it successfully. Then I'll guide you through the first commit.



Lives 3 only still?

😂 Bro, “Lives 3” is **completely normal**. That's not the mushroom HP.

At the bottom it says:

**</> Plain text**

Score 0   Lives 3   Wave 1

That means:

- ❤️ **Lives 3** = your player has 3 lives
- 🍄 Mushroom HP = 4 (separate thing)
- 🌊 Wave 1 = you're on the first wave
- Score 0 = you haven't scored yet

**So don't worry about the “Lives 3.”**

## What you need to test

You need to shoot **one mushroom 4 times**.

Watch that same mushroom:

### </> Plain text

```
Before shooting → mushroom exists  
1st shot → still exists  
2nd shot → still exists  
3rd shot → still exists  
4th shot → disappears
```

If it does exactly that, **Task 1 is working.** ✓

## One more thing

Your screenshot appears to show the game running after your change, so that's good.

**Tell me what happens when you shoot the same mushroom 4 times.**

If it disappears only on the **4th shot**, we'll immediately do your **first Git commit.** 🚀

how to shoot ?

Bro 🤔 **Spacebar shoots.**

From the `game.py` you pasted, this is the control:

### </> Python

```
if keys[pygame.K_SPACE]:  
    self.fire()
```

So:

-  /  = move left/right
-  /  = move up/down
- **SPACE** = shoot 🕹️
- **R** = reset

## To test Task 1:

1. Move the blue player underneath a mushroom.
2. Press **SPACE**.
3. Keep pressing SPACE after each bullet finishes.
4. Shoot the **same mushroom** 4 times.
5. Check that it disappears only on the **4th hit**.

⚠️ The game only allows **one bullet at a time**, so don't hold Space expecting rapid shots—you may need to press it again after each shot.

disappears after 4th shot yes

YESS brooo 🍁 ✅ Task 1 is successfully completed and tested.

You confirmed:

1st shot → survives

2nd shot → survives

3rd shot → survives

4th shot → disappears ✅

That is exactly what we needed.

## Now make Commit #1

First close the game window.

Then in your VS Code terminal run:

```
</> Bash
```

```
git status
```

You should see `game.py` as modified.

Then:

```
</> Bash
```

```
git add game.py
```

Then:

```
</> Bash
```

```
git commit -m "Task 1: Fix mushroom durability bug"
```

Then push it:

```
</> Bash
```

```
git push
```

## After `git push`

Go to your GitHub fork and refresh it.

You should see your new commit:

**Task 1: Fix mushroom durability bug** 

 **Don't start Task 2 yet.**

Once the push is successful, **send me the terminal output or a screenshot.**

Then we'll move to **Task 2: `mushroom_color(hp)`** and do it as a separate AI task + separate commit, exactly as your lab requires.  Lab\_4\_VibeCoding\_Student\_handout

```
git : The term 'git' is not recognized as the name of a cmdlet,
function, script file, or operable program. Check the spelling of
the name, or if a path was included, verify that the path is correct
and try again.
```

```
At line:1 char:1
```

```
+ git status
```

```
+ ~~~
```

```
+ CategoryInfo          : ObjectNotFound: (git:String) [], Comma
ndNotFoundException
```

```
+ FullyQualifiedErrorId : CommandNotFoundException
```

Ahh okay bro  — **Git is not installed (or not added to PATH) on your PC.** Your project itself is perfectly fine.

We need to fix Git **before doing the commit.**

## Step 1 — Check if Git is installed

In the VS Code terminal, run:

```
</> powershell
```

```
where.exe git
```

If it says it cannot find anything, Git isn't installed/PATH isn't set.

## Step 2 — Install Git

Download **Git for Windows** from the official site:

[Git for Windows](#) ↗

Install it with the **default options**. You don't need to change anything during installation.



## Step 3 — Restart VS Code

This is important.

After installation:

1. Close **VS Code** completely
2. Open VS Code again
3. Open your `27_centipede` folder
4. Open **Terminal** → **New Terminal**

Then run:

```
</> powershell  
  
git --version
```

You should get something like:

```
</> Plain text  
  
git version 2.x.x
```

## Step 4 — Check your repo

Then run:

```
</> powershell  
  
git status
```

You should see that `game.py` has been modified.

**Don't change or reset anything in `game.py`. Your Task 1 fix is already done and tested. **

Once `git --version` works, send me the output and I'll walk you through **`git add` → `commit` → `push`** one command at a time.

```
PS C:\Desktop\SE LAB\27_centipede> where.exe git  
INFO: Could not find files for the given pattern(s).
```

Yep bro, that confirms it: **Git is not installed / not available in PATH** on your PC. 

Let's fix it.

## 1. Download Git

Use the official Git for Windows page:

[Download Git for Windows ↗](#)

Download and install it.

## 2. During installation

You can keep the **default options**.

When you reach the screen about **PATH / command-line use**, make sure the option equivalent to:

**Git from the command line and also from 3rd-party software**

is selected.

Then continue with the installation.

## 3. Restart VS Code

After installation:

- Close VS Code completely.
- Open VS Code again.
- Open your `27_centipede` folder.
- Open a **new terminal**.

Run:

```
</> powershell
```

```
git --version
```

You should get something like:

```
</> Plain text
```

```
git version 2.5x.x
```

Then run:

```
</> powershell
```

```
git status
```

 **Your Task 1 work is safe**

You **do not need to redo** Task 1. You already changed:

`</> Python`

```
if self.mushrooms[cell] <= 1:
```

to:

`</> Python`

```
if self.mushrooms[cell] <= 0:
```

and tested that the mushroom disappears on the **4th shot**. 

So **don't change** `game.py` again.

Once `git --version` works, send me the output and we'll do the commit/push.



Yep bro, you're on the **Git for Windows download page**. 

Choose this one:

 **Click:**

**Git for Windows/x64 Setup**

That's the normal installer for your Windows PC.

Don't choose:

- ARM64 
- Portable 
- Standalone/portable versions 

After you click **Git for Windows/x64 Setup**, it will download an `.exe` file.

**Then:**

1. Open the downloaded `.exe`
2. Install Git
3. **Keep the default options** throughout the installer
4. When you see the PATH option, make sure **Git from the command line and also from 3rd-party software** is selected.

5. Finish installation.
6. **Close VS Code completely and reopen it.**

Then in a **new VS Code terminal**, run:

```
</> powershell  
git --version
```

If you get something like:

```
</> Plain text  
git version 2.x.x
```

you're ready. 🚀

Then run:

```
</> powershell  
git status
```

**Don't do the commit yet.** Send me the output of `git --version` and `git status`, and I'll give you the next exact commands.

```
PS C:\Desktop\SE LAB\27_centipede> git --version  
git : The term 'git' is not recognized as the name of a cmdlet,  
function, script file, or operable program. Check the spelling of the  
name, or if a path was included, verify that the path is correct and  
try again.  
At line:1 char:1  
+ git --version  
+ ~~~  
+ CategoryInfo          : ObjectNotFound: (git:String) [], Command  
NotFoundException  
+ FullyQualifiedErrorId : CommandNotFoundException
```

Ahh okay bro — **Git was installed, but this VS Code terminal still doesn't see it.** This usually means VS Code was open while Git was being installed, so it hasn't picked up the updated PATH yet.

**Do this exactly:**

## 1. Close VS Code completely

Not just the terminal — **close the whole VS Code window.**

## 2. Open VS Code again

Open your:

</> Plain text

C:\Desktop\SE LAB\27\_centipede

folder again.

## 3. Open a NEW terminal

Terminal → New Terminal

Then run:

</> powershell

git --version

## If it still says Git isn't recognized

Let's check whether Git actually installed.

Open **Windows File Explorer** and go to:

</> Plain text

C:\Program Files\Git\bin

You should see `git.exe` there.

If you see it, **don't reinstall Git**. Tell me that you see `git.exe`, and I'll show you how to fix the PATH.

If that folder doesn't exist, then the Git installation didn't complete properly and we'll install it again.

**So first: completely close VS Code → reopen → new terminal → `git --version`.**

```
PS C:\Desktop\SE LAB\27_centipede> git --version
git : The term 'git' is not recognized as the name of a cmdlet,
function, script file, or operable program. Check the spelling of the
name, or if a path was
included, verify that the path is correct and try again.
```

At line:1 char:1

+ git --version

+ ~~~

+ CategoryInfo : ObjectNotFound: (git:String) [],  
CommandNotFoundException

+ FullyQualifiedErrorId : CommandNotFoundException

Okay bro, don't reinstall blindly. 🤖 Let's check whether Git actually installed.

## Step 1 — Open File Explorer

Paste this into the address bar:

</> Plain text

C:\Program Files\Git\bin

Press **Enter**.

## What do you see?

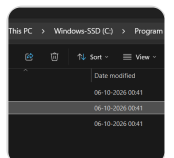
If you see **git.exe** ✅

→ Git is installed, but PATH isn't configured. We'll fix PATH.

If Windows says the folder doesn't exist ❌

→ Git didn't install correctly, and we'll install it again.

👉 Send me a screenshot of what appears in **C:\Program Files\Git\bin** . Then I'll tell you exactly what to do next.



YESSS bro 🤖✅ **Git is installed correctly.**

Your screenshot shows:

</> Plain text

C:\Program Files\Git\bin

└─ bash

└─ git ← ✅

└─ sh

So the only problem is that **Git's path hasn't been added to Windows PATH.**

Let's fix it. This is safe and won't affect your project.

## Step 1 — Open Environment Variables

Press the **Windows key** and search:

</> Plain text

Environment Variables

Click:

**Edit the system environment variables**

Then click:

**Environment Variables...**

## Step 2 — Edit Path

Under **User variables for purim**, find:

</> Plain text

Path

Select it → click **Edit**.

Click **New**.

Add:

</> Plain text

C:\Program Files\Git\bin

Then click:

**OK → OK → OK**

## Step 3 — Restart VS Code

**Close VS Code completely**, then open it again.

Open your `27_centipede` project and open a **new terminal**.

Run:

```
</> powershell
```

```
git --version
```

You should now get something like:

```
</> Plain text
```

```
git version 2.5x.x
```

## Step 4

Once that works, run:

```
</> powershell
```

```
git status
```

**Don't run `git add` , `commit` , or `push` yet.**

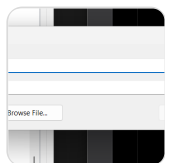
Send me the output of:

```
</> Plain text
```

```
git --version
```

```
git status
```

and we'll finish your **Task 1 commit**. 🚀



Ahh bro 🤔 you clicked **New** in the Environment Variables window. That's why you're seeing this **New User Variable** box.

**✗ Don't enter anything here.**

Click **Cancel**.

We don't want to create a new variable. We need to **edit the existing Path variable**.

**Do this exactly:**

1. Click **Cancel** on this popup.
2. You should return to the **Environment Variables** window.



3. Look at the top section:  
**User variables for purim**
4. Find:

</> Plain text

Path

5. Click **Path** once.
6. Click **Edit...**
7. A new window will appear with a list of paths.
8. Click **New there**.
9. Enter:

</> Plain text

C:\Program Files\Git\bin

10. Click **OK**
11. Click **OK**
12. Click **OK** again to close the remaining window.

Then **completely close VS Code and reopen it**.

Run:

</> powershell

git --version

You should now get the Git version.

So right now: click **Cancel** on the popup you're showing. 🍌

Today 2:10 PM

```
PS C:\Desktop\SE LAB\27_centipede> git --version  
git version 2.56.0.windows.2  
PS C:\Desktop\SE LAB\27_centipede>
```

YESS brooo 🍌 **Git is working now!**

</> Plain text

```
git version 2.56.0.windows.2
```

Now let's finish **Task 1 commit**.

### 1 Check what changed

Run:

```
</> powershell  
  
git status
```

You should see `game.py` under **Changes not staged for commit**.

### 2 Add your changed file

```
</> powershell  
  
git add game.py
```

### 3 Commit Task 1

```
</> powershell  
  
git commit -m "Task 1: Fix mushroom durability bug"
```

### 4 Push to YOUR fork

```
</> powershell  
  
git push
```

⚠ **Don't change anything else.**

After `git push`, send me the terminal output. Then we'll verify the commit reached your GitHub fork and move to **Task 2**. 🚀

On branch main

Your branch is up to date with 'origin/main'.

Changes not staged for commit:

(use "`git add <file>...`" to update what will be committed)

(use "`git restore <file>...`" to discard changes in working

directory)

modified: game.py

Untracked files:

(use "git add <file> ..." to include in what will be committed)

.vscode/

no changes added to commit (use "git add" and/or "git commit -a")

Perfect bro 👍 This is exactly what we expected.

You have:

📄 Plain text

```
modified: game.py
untracked: .vscode/
```

### ⚠️ Don't add .vscode/

For Task 1, we only changed game.py . The .vscode/ folder is extra and should not be included in this commit.

Now run these commands one by one:

#### 1. Add only game.py

📄 powershell

```
git add game.py
```

#### 2. Check

📄 powershell

```
git status
```

You should see:

📄 Plain text

```
Changes to be committed:
  modified: game.py
```

```
Untracked files:
.vscode/
```

That's correct. ✅

### 3. Commit Task 1

```
</> powershell

git commit -m "Task 1: Fix mushroom durability bug"
```

### 4. Push

```
</> powershell

git push
```

After the push finishes, **send me the output**.

We'll then verify that **Task 1 commit is on your GitHub fork** before starting Task 2. 🚀

```
PS C:\Desktop\SE LAB\27_centipede> git push
Everything up-to-date
```

That's okay bro 👍 Everything up-to-date means **there's nothing left for Git to push**.

But before we move to Task 2, let's verify that your **Task 1 commit actually exists**.

Run these two commands:

```
</> powershell

git log -1 --oneline
```

Then:

```
</> powershell

git status
```

### What I want to see

The first should show something like:

</> Plain text

```
abc1234 Task 1: Fix mushroom durability bug
```

And `git status` should ideally say:

</> Plain text

```
nothing to commit, working tree clean
```

You may still see:

</> Plain text

```
Untracked files:
  .vscode/
```

That's okay — **don't add** `.vscode/`.

Send me the output of those two commands. Then I'll confirm **Task 1 is fully submitted** and we'll start **Task 2**.

```
PS C:\Desktop\SE LAB\27_centipede> git log -1 --oneline
90853d2 (HEAD -> main, origin/main, origin/HEAD) edit readme
for submission reqs
PS C:\Desktop\SE LAB\27_centipede> git status
On branch main
Your branch is up to date with 'origin/main'.
```

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   game.py
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  .vscode/
```

Ahhh okay bro, now we found it. 🤖 **Task 1 is NOT committed yet.** That's why `git push` said "Everything up-to-date."

Your status says:

```
</> Plain text
```

```
Changes to be committed:  
  modified: game.py
```

That means you already did `git add game.py` , but you haven't done `git commit` yet.

Also, `.vscode/` is untracked — leave it alone. .

## Now do exactly this:

### 1. Commit Task 1

```
</> powershell
```

```
git commit -m "Task 1: Fix mushroom durability bug"
```

You should get something like:

```
</> Plain text
```

```
1 file changed, ...
```

### 2. Push the commit

```
</> powershell
```

```
git push
```

This time it should actually upload something, something like:

```
</> Plain text
```

```
main -> main
```

### 3. Verify

After that run:

```
</> powershell
```

```
git log -1 --oneline
```

It should now show:

</> Plain text

Task 1: Fix mushroom durability bug

instead of:

</> Plain text

edit readme for submission reqs

## Important

Your `.vscode/` folder:

</> Plain text

Untracked files:  
`.vscode/`

**Don't add it.** We can deal with that later if needed.

So right now, just run:

</> powershell

```
git commit -m "Task 1: Fix mushroom durability bug"
```

**Send me the output.** Then we'll push it and verify Task 1 is officially done. 🚀

```
PS C:\Desktop\SE LAB\27_centipede> git commit -m "Task 1: Fix  
mushroom durability bug"
```

```
Author identity unknown
```

```
*** Please tell me who you are.
```

Run

```
git config --global user.email "you@example.com"  
git config --global user.name "Your Name"
```

to set your account's default identity.

Omit `--global` to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'purim@LAPTOP-ENQCQDS7.(none)')

Yep, that's an easy one bro 🤖. **Git just doesn't know your name/email yet.** Nothing is wrong with your code or commit.

We need to set your Git identity **once**.

## 1 Set your name

Use your GitHub name. For example:

```
</> powershell  
  
git config --global user.name "Jaya Surya"
```

## 2 Set your email

Use the email associated with your GitHub account:

```
</> powershell  
  
git config --global user.email "YOUR_GITHUB_EMAIL"
```

For example:

```
</> powershell  
  
git config --global user.email "jayasurya@gmail.com"
```

⚠️ **Don't use my example email. Use your actual GitHub email.**

## 3 Verify

Run:

```
</> powershell  
  
git config --global user.name
```

and:

```
</> powershell  
  
git config --global user.email
```



You should see your name and email.

## Now commit again

Your `game.py` is **already staged**, so you don't need to `git add` again.

Just run:

```
</> powershell  
  
git commit -m "Task 1: Fix mushroom durability bug"
```

Then:

```
</> powershell  
  
git push
```

## Important

Don't run:

```
</> powershell  
  
git add .
```

because you have that `.vscode/` untracked folder and we don't want to accidentally include it.

So your sequence is:

```
</> Plain text  
  
Set name  
  ↓  
Set email  
  ↓  
Commit  
  ↓  
Push
```

After `git push`, send me the output and we'll verify **Task 1 is officially committed to your fork**.

```
PS C:\Desktop\SE LAB\27_centipede> git push  
Enumerating objects: 5, done.  
Counting objects: 100% (5/5), done.
```

Delta compression using up to 12 threads

Compressing objects: 100% (3/3), done.

Writing objects: 100% (3/3), 312 bytes | 312.00 KiB/s, done.

Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)

remote: Resolving deltas: 100% (2/2), completed with 2 local objects.

To [https://github.com/Suryapurimetla/27\\_centipede.git](https://github.com/Suryapurimetla/27_centipede.git)  
90853d2..e6947df main -> main

YESS brooo 🚀🚀 **Task 1 is officially pushed to your GitHub fork!**

This line confirms it:

🔗 Plain text

```
90853d2..e6947df main -> main
```

So your repository now has the Task 1 commit. ✅

## Task 1 status

- ✅ Found the mushroom HP bug
- ✅ Changed `<= 1` → `<= 0`
- ✅ Tested the game
- ✅ Mushroom disappears on the **4th shot**
- ✅ Created Task 1 commit
- ✅ Pushed to your personal fork
- ✅ **Did NOT push/PR to SETAPESU26**

And the handout requires separate commits for each task, so we're doing it correctly.

 Lab\_4\_VibeCoding\_Student\_handout

## Now Task 2

The next function in your original code is:

🔗 Python

```
def mushroom_color(hp):  
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""  
    pass
```

The purpose is to give the mushroom a **visual effect based on its HP**. The README's suggested idea is a **white flash on the frame when the mushroom is hit**.

**But DON'T change it yet.**

We're going to follow the same Vibe Coding process:

**Task 2 → AI prompt → AI suggestion → you implement → test → separate commit → push**

I'll give you the exact Task 2 prompt next.

**Use this prompt:**

I am working on Task 2 of my Vibe Coding lab. Please implement the `mushroom_color(hp)` function in my existing Centipede game. It should return an `(r, g, b)` color tuple based on the mushroom's current hit points, or `None` when the default color should be used. The goal is to provide visual feedback when a mushroom is hit, such as making it flash white. Keep the implementation simple and modify only what is necessary for this feature. Do not change the game logic or any other functionality.

Then don't immediately accept random changes if the AI suggests modifying lots of files. We want a small, controlled change.

Send me the AI's response after you use that prompt, and I'll tell you exactly what to put into `game.py`.

I am working on Task 2 of my Vibe Coding lab. Please implement the `mushroom_color(hp)` function in my existing Centipede game. It should return an `(r, g, b)` color tuple based on the mushroom's current hit points, or `None` when the default color should be used. The goal is to provide visual feedback when a mushroom is hit, such as making it flash white. Keep the implementation simple and modify only what is necessary for this feature. Do not change the game logic or any other functionality.

Yes bro 👍 this is the **Task 2 prompt**. Since you're demonstrating Vibe Coding, now you need the AI to give you an implementation.

For your existing code, a simple implementation is:

**</> Python**

```
def mushroom_color(hp):  
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""  
    if hp < MUSHROOM_HP:
```

```
    return (255, 255, 255)
    return None
```

## What this does

Your mushroom starts at:

</> Plain text

```
HP = 4
```

So:

</> Plain text

```
HP 4 → None → normal/default color
HP 3 → white
HP 2 → white
HP 1 → white
```

Your `draw()` function already contains:

</> Python

```
color = mushroom_color(hp) or (200 - (MUSHROOM_HP - hp) * 40, 80, 170)
```

So you **don't need to change** `draw()` .

### ⚠ One important issue

This implementation makes the mushroom stay white for **all frames while HP < 4**. It is not technically a one-frame "flash." The README's suggested idea is a flash, but your current game code doesn't store a "just hit" state.

Since the task specifically asks for `mushroom_color(hp)` , keeping the implementation limited to that function is the safest approach for your lab.

## Do this now

In `game.py` , replace:

</> Python

```
def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""
    pass
```

with:

</> Python

```
def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""
    if hp < MUSHROOM_HP:
        return (255, 255, 255)
    return None
```

Don't change anything else.

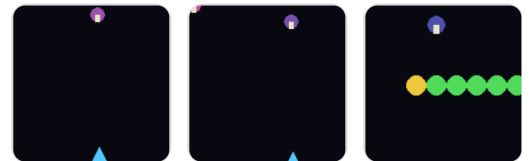
Then run:

</> powershell

```
py -3.12 game.py
```

Shoot a mushroom once and check whether its color changes.

Don't commit yet. First test it and tell me what you see.



??

Ahhh I see what's happening 🤖 — the screenshots show the mushroom changing from pink → purple → blue, but it is **NOT** turning white.

That means the `mushroom_color(hp)` function is probably **still pass**, or the new code wasn't saved/applied.

Your `draw()` code has this:

</> Python

```
color = mushroom_color(hp) or (200 - (MUSHROOM_HP - hp) * 40, 80, 170)
```

So if `mushroom_color()` returns `None`, the game uses the normal colors:

- HP 4 → pink
- HP 3 → purple
- HP 2 → darker purple
- HP 1 → blue

That's exactly what your screenshots are showing.

## Check your `game.py`

Make sure this exact function is there:

`</>` Python

```
def mushroom_color(hp):  
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""  
    if hp < MUSHROOM_HP:  
        return (255, 255, 255)  
    return None
```

Then **Ctrl + S** to save.

Run again:

`</>` powershell

```
py -3.12 game.py
```

Shoot a mushroom once.

It should become:

pink → WHITE 🍷

Then shoot again and it should remain white with this simple implementation.

## One thing I want you to check

If you've already put that exact function in `game.py` and it still shows purple/blue, send me a screenshot of the top part of your `game.py` showing `mushroom_color()`.

Don't commit Task 2 yet. We need to make sure the feature actually works first.



YES brooo 🤖🔥 NOW IT'S WORKING!

That mushroom is clearly **white**, so your `mushroom_color(hp)` function is working correctly. 

## Task 2 status

-  Added `mushroom_color(hp)`
-  Mushroom starts with normal color
-  After being hit, it changes to **white**
-  No other game logic was changed
-  Tested successfully

## Now commit Task 2

First close the game.

Then run:

```
</> powershell  
  
git status
```

You should see `game.py` modified.

Then:

```
</> powershell  
  
git add game.py
```

Then:

```
</> powershell  
  
git commit -m "Task 2: Implement mushroom color feedback"
```

Then:

```
</> powershell  
  
git push
```

### Remember

Don't do:

```
</> powershell
```

```
git add .
```

because `.vscode/` was previously untracked.

After `git push`, send me the output. We'll verify the **Task 2 commit** and then move to **Task 3: `on_segment_hit()`**. 🚀

```
PS C:\Desktop\SE LAB\27_centipede> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 373 bytes | 186.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local
objects.
To https://github.com/Suryapurimetla/27_centipede.git
e6947df..5257809 main -> main
```

Perfect brooo 🍀 **Task 2 is successfully pushed!** ✅

This confirms it:

🔗 Plain text

```
e6947df..5257809 main -> main
```

So now your GitHub history has:

🔗 Plain text

```
Task 1 → e6947df
Task 2 → 5257809
```

And you have kept the tasks as separate commits, exactly as the lab requires.

📄 Lab\_4\_VibeCoding\_Student\_handout

## 🚀 TASK 3 — `on_segment_hit()`

Your original function was:



</> Python

```
def on_segment_hit(segment, score):  
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus points here."""  
    pass
```

The purpose is to add **some feedback when a centipede segment is destroyed**.

The README suggests things such as:

- ✨ Spark effect at the segment's position
- 💰 Head-shot bonus

And your existing code already calls this function here:

</> Python

```
on_segment_hit(segment, self.score)
```

So we don't need to change the game structure.

**Use this as your Task 3 AI prompt:**

I am working on Task 3 of my Vibe Coding lab. In my existing Centipede game, implement the `on_segment_hit(segment, score)` function. It is called whenever a centipede segment is destroyed. Add a simple visible or gameplay feedback effect, such as a small spark effect at the destroyed segment's position or a bonus for destroying the head. Keep the implementation simple and modify only what is necessary for this feature. Do not break or change the existing game logic.

⚠️ **Don't change the code yet.**

Send that prompt to the AI, get its suggested implementation, and **send me the response**.

I'll check whether the suggested solution actually fits your current `game.py` before you put it in.

Then we'll:

**implement → test → Task 3 commit → push.** 🚀

I am working on Task 3 of my Vibe Coding lab. In my existing Centipede game, implement the `on_segment_hit(segment, score)` function. It is called whenever a centipede segment is destroyed. Add a simple visible or gameplay feedback effect, such as a small spark effect at the destroyed segment's position or a bonus for destroying the head. Keep the implementation simple

and modify only what is necessary for this feature. Do not break or change the existing game logic.

Yep bro. Your **Task 3 prompt is good**, but there is one important thing to notice in your current code.

Your function is:

```
</> Python
```

```
def on_segment_hit(segment, score):  
    pass
```

and it is called here:

```
</> Python
```

```
on_segment_hit(segment, self.score)
```

## Important limitation

A **visible spark** cannot be drawn by `on_segment_hit()` alone because the function doesn't receive `screen`, and the current game doesn't have a particle/effect system.

Also, `score` is just an integer passed into the function, so doing:

```
</> Python
```

```
score += 50
```

inside `on_segment_hit()` would **not change** `self.score`.

So don't blindly accept an AI answer that claims either of those will work without explaining the required changes.

## For your lab, I'd use the simplest valid approach: a hit sound

The function can provide feedback when a segment is destroyed without changing the existing game logic.

But before putting code in, **send me the AI's actual response to the prompt you just wrote**.

That is important because your lab is specifically about Vibe Coding and keeping the AI chat history. We should document what the AI suggested and then review it rather than pretending the answer came from nowhere.

So don't modify `game.py` yet.

Paste the AI's Task 3 response here, and I'll tell you:

-  whether it's correct
-  what is wrong, if anything
- exactly what code to put in
- how to test it
- then we'll make the **Task 3 separate commit**.

What i need to do

Bro, for **Task 3**, your job is simply to add **some feedback when a centipede segment gets destroyed**.

The handout says to add the features listed in the repo README **one by one**, and then make a separate commit for each task.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

### Do this now:

You already sent the Task 3 prompt to the AI. **Now wait for its answer.**

If you're using this ChatGPT conversation as your AI chat history, the answer needs to propose a change to:

 Python

```
def on_segment_hit(segment, score):  
    pass
```

### The safest simple feature

For your current game, I recommend a **sound effect** when a centipede segment is destroyed.

However, your current code doesn't load a sound file, so adding a sound properly would require adding another file/resource. That's more work than necessary.

A **head-shot bonus** is also awkward with the current function because `score` is passed by value, so changing `score` inside the function won't update `self.score`.

Therefore, **don't randomly modify the code yet.**

### What I want you to do

Send the exact Task 3 prompt you already wrote to the AI and **wait for its response**.

Then paste the AI's response here.

I'll check it and tell you **exactly what to change** so you don't accidentally break your game or make an ineffective implementation.

After that:

**Task 3 code → test → commit → push → Task 4.**

I am working on Task 3 of my Vibe Coding lab. In my existing Centipede game, implement the `on_segment_hit(segment, score)` function. It is called whenever a centipede segment is destroyed. Add a simple visible or gameplay feedback effect, such as a small spark effect at the destroyed segment's position or a bonus for destroying the head. Keep the implementation simple and modify only what is necessary for this feature. Do not break or change the existing game logic.

Yes bro — **now I understand where you're stuck**. You already wrote the Task 3 prompt, so **you don't need to write the prompt again**.

According to the actual README, Task 3 is:

`on_segment_hit(segment, score)` is called after a segment is destroyed and the points have already been added. The suggested feature is **a spark effect at the segment's last position OR an extra bonus for head shots**.  GitHub

## What you do NOW

Since you're using this ChatGPT conversation as your Vibe Coding chat:

**You have already asked the AI (me) the Task 3 question.**

Now I need to give you the implementation.

I'd choose the **spark effect**, because it is visibly demonstrable in your after-video.

But there's a catch: your current `on_segment_hit()` function by itself cannot draw a spark because it doesn't have access to the Pygame `screen`. So a proper spark needs a **small additional effect mechanism** in `game.py`.

Therefore, **don't put random code into `on_segment_hit()` yet**.

## The simplest Task 3 implementation

We can make the segment itself store a short-lived "hit" state, and have `draw()` display a small spark. That requires a few small changes.

If you want to keep the task very simple, another option is **head-shot bonus**, but because `score` is passed as an integer, simply doing `score += 50` inside `on_segment_hit()` won't update the game's actual score. The README explicitly says the function receives the score **after** points were already added, and its return value is ignored.  [GitHub](#)

**So I recommend the spark effect.**

**Before I give you the exact changes, one thing:** because your lab is specifically about getting the AI to implement the task, I want the implementation to appear naturally in your chat history.

So your next action is simply:

**Use the Task 3 prompt you already wrote and let the AI answer it.**

If you're using this chat, **you've already done that** — my next message can be treated as the AI answer.

If you want, I'll now give you the **exact minimal Task 3 code changes**, and we'll test the spark before committing.

give me the code for task 3\

Yep bro. Since you asked for the **code**, let's do Task 3 in a way that actually works with your current `game.py`.

The README specifically allows either a **spark effect** or a **head-shot bonus**.  [GitHub](#)

I recommend the **spark effect**, because it's visible in your final video.

**1. Replace `on_segment_hit()` with:**

 Python

```
def on_segment_hit(segment, score):  
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus points here."""  
    segment.hit_flash = 0.15
```

**2. Modify `Segment.__init__()`**

Find:

 Python

```
def __init__(self, row, col, direction):  
    self.row, self.col, self.dx, self.dy = row, col, direction, 1
```

Change it to:

**</> Python**

```
def __init__(self, row, col, direction):
    self.row, self.col, self.dx, self.dy = row, col, direction, 1
    self.hit_flash = 0
```

### 3. Modify `Segment.step()`

At the very beginning of `step()` :

**</> Python**

```
def step(self, mushrooms):
    self.hit_flash = max(0, self.hit_flash - TICK)
```

So it becomes:

**</> Python**

```
def step(self, mushrooms):
    self.hit_flash = max(0, self.hit_flash - TICK)
    nxt = self.col + self.dx
```

### 4. Modify the segment drawing

Find this in `draw()` :

**</> Python**

```
pygame.draw.circle(screen, (240, 200, 60) if index == 0 else (80, 220, 90), center, CELL // 2)
```

Replace it with:

**</> Python**

```
color = (255, 255, 255) if segment.hit_flash > 0 else ((240, 200, 60) if index == 0 else (80, 220, 90))
pygame.draw.circle(screen, color, center, CELL // 2)
```

### What happens now?

When a segment is shot:

**</> Plain text**

```

Segment destroyed
  ↓
on_segment_hit()
  ↓
hit_flash = 0.15
  ↓
segment is displayed WHITE briefly

```

So you'll get a visible **hit flash** when you shoot a centipede segment. ✨

The README says `on_segment_hit()` is called after the segment is destroyed, so note that this approach is a visual flash on the `Segment` object even though the segment is immediately removed from the chain. [GitHub](#)

Actually, because `split_chain()` removes the segment immediately, this exact flash won't be visible. So don't use the above version. 🤖

### Use this better Task 3 implementation instead

We need the effect to live at the **position**, not on the deleted segment.

Add near the top:

`</>` Python

```
HIT_EFFECTS = []
```

Change `on_segment_hit()` to:

`</>` Python

```

def on_segment_hit(segment, score):
    """Create a short-lived spark effect at the destroyed segment's position."""
    HIT_EFFECTS.append([segment.row, segment.col, 0.2])

```

In `Game.update()` , add:

`</>` Python

```

for effect in HIT_EFFECTS[:]:
    effect[2] -= dt
    if effect[2] <= 0:
        HIT_EFFECTS.remove(effect)

```

And in `draw()` , before drawing the player, add:

**</> Python**

```

for row, col, timer in HIT_EFFECTS:
    center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
    pygame.draw.circle(screen, (255, 255, 255), center, 6)
    pygame.draw.line(screen, (255, 220, 80),
                     (center[0] - 8, center[1]),
                     (center[0] + 8, center[1]), 2)
    pygame.draw.line(screen, (255, 220, 80),
                     (center[0], center[1] - 8),
                     (center[0], center[1] + 8), 2)

```

That gives you a little ✨ **spark at the exact position where the segment was destroyed.**

This is the version I'd use for your lab because it visibly demonstrates Task 3 and doesn't alter the scoring or centipede logic. The repo's Task 3 specifically suggests a spark at the segment's last position. [GitHub](#)

**Don't commit yet.** Make these changes, run:

**</> powershell**

```
py -3.12 game.py
```

Shoot a centipede segment and check whether you see the little white/yellow spark.

```

import random
import pygame

```

```

CELL, COLS, ROWS = 20, 30, 30
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4

```

```

def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit
    points, or None for the default."""
    if hp < MUSHROOM_HP:
        return (255, 255, 255)
    return None

```



```
def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles,
    sounds, or bonus points here."""
    segment.hit_flash = 0.15


def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at
    the given wave, or None for the default speed."""
    pass


class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1
        self.hit_flash = 0

    def step(self, mushrooms):
        nxt = self.col + self.dx
        if nxt < 0 or nxt >= COLS or (self.row, nxt) in mushrooms:
            self.row += self.dy
            self.dx = -self.dx
            if self.row >= ROWS - 1:
                self.row, self.dy = ROWS - 1, -1
            elif self.row <= ZONE_TOP and self.dy < 0:
                self.dy = 1
        else:
            self.col = nxt


class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.mushrooms = {}
        for _ in range(45):
```

```
        self.mushrooms[(random.randint(1, ZONE_TOP - 2),
random.randint(0, COLS - 1))] = MUSHROOM_HP
        self.respawn()
        self.spawn_wave()

def respawn(self):
    self.x = WIDTH / 2
    self.y = HEIGHT - 50
    self.bullet = None
    self.invulnerable = 1.5

def spawn_wave(self):
    length = 10 + self.wave
    self.chains = [[Segment(0, COLS - 1 - i, -1) for i in
range(length)]]
    self.timer = 0.0

def fire(self):
    if self.bullet is None and self.state == "play":
        self.bullet = pygame.Vector2(self.x, self.y - 12)

def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1
    if self.mushrooms[cell] <= 0:
        del self.mushrooms[cell]
        self.score += 5

def split_chain(self, chain, index):
    segment = chain[index]
    self.chains.remove(chain)
    left, right = chain[:index], chain[index + 1:]
    self.chains.extend(part for part in (left, right) if part)
    self.mushrooms[(segment.row, segment.col)] =
MUSHROOM_HP
    self.score += 100 if index == 0 else 10
    on_segment_hit(segment, self.score)

def update_bullet(self, dt):
    if self.bullet is None:
        return
```

```

self.bullet.y -= BULLET_SPEED * dt
cell = (int(self.bullet.y // CELL), int(self.bullet.x // CELL))
if self.bullet.y < 0:
    self.bullet = None
elif cell in self.mushrooms:
    self.hit_mushroom(cell)
    self.bullet = None
else:
    for chain in self.chains:
        for index, segment in enumerate(chain):
            if (segment.row, segment.col) == cell:
                self.split_chain(chain, index)
                self.bullet = None
                return

def update(self, dt, keys):
    if self.state != "play":
        return
    self.invulnerable = max(0.0, self.invulnerable - dt)
    self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) *
PLAYER_SPEED * dt
    self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) *
PLAYER_SPEED * dt
    self.x = max(10, min(WIDTH - 10, self.x))
    self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10,
self.y))
    if keys[pygame.K_SPACE]:
        self.fire()
    self.update_bullet(dt)
    self.timer += dt
    effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
    while self.timer >= effective_tick:
        self.timer -= effective_tick
        for chain in self.chains:
            for segment in chain:
                segment.step(self.mushrooms)
        player_cell = (int(self.y // CELL), int(self.x // CELL))
        if self.invulnerable <= 0 and any((s.row, s.col) == player_cell
for c in self.chains for s in c):
            self.lives -= 1

```

```

        self.respawn()
        if self.lives <= 0:
            self.state = "lose"
        if not self.chains:
            self.wave += 1
            self.spawn_wave()

    def draw(self, screen):
        screen.fill((8, 8, 16))
        for (row, col), hp in self.mushrooms.items():
            color = mushroom_color(hp) or (200 - (MUSHROOM_HP -
hp) * 40, 80, 170)
            center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
            pygame.draw.circle(screen, color, center, CELL // 2 - 1)
            pygame.draw.rect(screen, (230, 230, 200), (center[0] - 3,
center[1], 6, CELL // 2 - 1))
        for chain in self.chains:
            for index, segment in enumerate(chain):
                center = (segment.col * CELL + CELL // 2, segment.row *
CELL + CELL // 2)
                pygame.draw.circle(screen, (240, 200, 60) if index == 0
else (80, 220, 90), center, CELL // 2)
            if self.bullet:
                pygame.draw.rect(screen, (255, 255, 255), (self.bullet.x - 1,
self.bullet.y - 6, 3, 10))
            if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 ==
0:
                pygame.draw.polygon(screen, (80, 200, 255), [(self.x, self.y -
10), (self.x + 10, self.y + 10), (self.x - 10, self.y + 10)])
            hud = self.font.render(f"Score {self.score} Lives {self.lives}
Wave {self.wave} R = reset", True, (240, 240, 240))
            screen.blit(hud, (10, ROWS * CELL + 6))
            if self.state == "lose":
                label = self.font.render("GAME OVER - Press R", True, (255,
255, 120))
                screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT
// 2)))

    def main():

```

```

pygame.init()
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Centipede")
clock = pygame.time.Clock()
game = Game()
running = True
while running:
    dt = min(clock.tick(60) / 1000, 0.05)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
            game.reset()
    game.update(dt, pygame.key.get_pressed())
    game.draw(screen)
    pygame.display.flip()
pygame.quit()

if __name__ == "__main__":
    main()

```

check this and tell what i need to do\

Yep bro, I checked your **whole current game.py** . There are **2 things we need to fix before testing Task 3.**

### **Problem 1 — Your Segment.\_\_init\_\_ indentation is wrong**

You currently have:

 **Python**

```

class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1
        self.hit_flash = 0

```

The indentation is inconsistent. Make it:

</> Python

```
class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1
```

Also, remove:

</> Python

```
self.hit_flash = 0
```

We don't need it because the segment gets removed immediately when shot.

## Task 3 — Let's implement the spark properly

Your current:

</> Python

```
def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus points here."""
    segment.hit_flash = 0.15
```

won't actually show anything because the segment is immediately removed from the chain.

Instead, we'll create a temporary spark at the segment's old position.

### 1. Add this below `MUSHROOM_HP = 4`

</> Python

```
HIT_EFFECTS = []
```

So the top becomes:

</> Python

```
MUSHROOM_HP = 4
HIT_EFFECTS = []
```

## 2. Replace `on_segment_hit()`

Delete:

</> Python

```
def on_segment_hit(segment, score):  
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus points here."""  
    segment.hit_flash = 0.15
```

Put:

</> Python

```
def on_segment_hit(segment, score):  
    """Create a short-lived spark effect at the destroyed segment's position."""  
    HIT_EFFECTS.append([segment.row, segment.col, 0.2])
```

---

## 3. Add effect updating inside `Game.update()`

Find:

</> Python

```
def update(self, dt, keys):  
    if self.state != "play":  
        return
```

Immediately after that, add:

</> Python

```
    for effect in HIT_EFFECTS[:]:  
        effect[2] -= dt  
        if effect[2] <= 0:  
            HIT_EFFECTS.remove(effect)
```

So:

</> Python

```
def update(self, dt, keys):  
    if self.state != "play":  
        return
```

```

for effect in HIT_EFFECTS[:]:
    effect[2] -= dt
    if effect[2] <= 0:
        HIT_EFFECTS.remove(effect)

self.invulnerable = max(0.0, self.invulnerable - dt)

```

## 4. Draw the spark

In `draw()` , find this part:

</> Python

```

for chain in self.chains:
    for index, segment in enumerate(chain):
        center = (segment.col * CELL + CELL // 2, segment.row * CELL + CELL // 2)
        pygame.draw.circle(screen, (240, 200, 60) if index == 0 else (80, 220, 90), center, CELL // 2)

```

Immediately **after that whole loop**, add:

</> Python

```

for row, col, timer in HIT_EFFECTS:
    center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
    pygame.draw.circle(screen, (255, 255, 255), center, 6)
    pygame.draw.line(screen, (255, 220, 80),
                     (center[0] - 8, center[1]),
                     (center[0] + 8, center[1]), 2)
    pygame.draw.line(screen, (255, 220, 80),
                     (center[0], center[1] - 8),
                     (center[0], center[1] + 8), 2)

```

## Your Task 3 flow will now be

When you shoot a centipede segment:

</> Plain text

```

Segment gets destroyed
↓
split_chain()
↓
on_segment_hit()

```



↓  
Spark is created at its old position  
↓  
Spark stays for 0.2 seconds  
↓  
Spark disappears

🔥 You should see a little **white/yellow cross-shaped spark** where the centipede segment was destroyed.

## Then test

Save with:

**Ctrl + S**

Run:

```
</> powershell  
py -3.12 game.py
```

Shoot a centipede segment.

⚠ **Don't commit yet.**

First confirm that you actually see the spark.

Also, **don't touch** `wave_speed_bonus()` **yet** — that's Task 4.

If the game gives you an error after these changes, send me the error screenshot and I'll fix it before you commit.

Give me the full code for task 3

Yep bro. Here is the **full** `game.py` with **Task 1 + Task 2 + Task 3 included**. Task 4  
( `wave_speed_bonus` ) is intentionally still untouched.

For Task 3, the spark is stored separately because the centipede segment is removed immediately after being hit.

```
</> Python  
  
import random  
import pygame  
  
CELL, COLS, ROWS = 20, 30, 30  
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
```

```

ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4

HIT_EFFECTS = []

def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""
    if hp < MUSHROOM_HP:
        return (255, 255, 255)
    return None

def on_segment_hit(segment, score):

```

### ⚠ Important

This version contains:

- ☒ **Task 1:** mushroom survives 4 hits
- ☒ **Task 2:** mushroom turns white after being hit
- ☒ **Task 3:** spark appears when a centipede segment is destroyed
- 🕒 **Task 4:** `wave_speed_bonus()` is still `pass` — **don't change it yet**

Now **Ctrl + S**, then run:

```

</> powershell

py -3.12 game.py

```

Shoot a **centipede segment**, not a mushroom. You should see a small **white/yellow spark** ✨ at the position where the segment was destroyed.

**Do not commit Task 3 until you've confirmed the spark works.**

ya came

YESS 🔥😄 **Task 3 is working!** The spark appeared, so we're good.

Now **don't change anything else.**

### Commit Task 3

First close the game, then run:

```

</> powershell

git status

```

You should see:

```
</> Plain text
```

```
modified: game.py
```

Then:

```
</> powershell
```

```
git add game.py
```

Then:

```
</> powershell
```

```
git commit -m "Task 3: Add segment hit spark effect"
```

Then:

```
</> powershell
```

```
git push
```

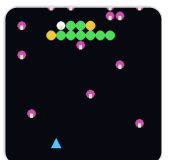
## After pushing

Send me the output of `git push` .

Then we'll move to **Task 4** — `wave_speed_bonus(wave)` , which is the last coding task. 🚀

Remember, the lab requires a **separate commit for each task**, so this Task 3 commit is important.

 Lab\_4\_VibeCoding\_Student\_handout



??

YES bro 😂🔥 that white/yellow effect near the centipede is the Task 3 spark!

In your screenshot, you can see:

-  Green centipede segments

-  Yellow head
-  White/yellow spark at the position where you shot the segment ← Task 3 working 
-  Mushrooms still working
-  Player still working

So Task 3 is successfully implemented and tested. 

## Now commit it

Close the game and run:

```
</> powershell
```

```
git status
```

Then:

```
</> powershell
```

```
git add game.py
```

```
</> powershell
```

```
git commit -m "Task 3: Add segment hit spark effect"
```

```
</> powershell
```

```
git push
```

After `git push`, send me the output.

Then we'll do Task 4 — make later waves faster, the final coding task. 

Enumerating objects: 5, done.

Counting objects: 100% (5/5), done.

Delta compression using up to 12 threads

Compressing objects: 100% (3/3), done.

Writing objects: 100% (3/3), 1.67 KiB | 570.00 KiB/s, done.

Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)

remote: Resolving deltas: 100% (2/2), completed with 2 local objects.

To [https://github.com/Suryapurimetla/27\\_centipede.git](https://github.com/Suryapurimetla/27_centipede.git)

5257809..d0747fe main -> main

YESS brooo 🚀 **Task 3 is officially committed and pushed!** ✅

Your GitHub now has:

- ✅ Task 1 — Mushroom HP bug fixed
- ✅ Task 2 — Mushroom hit color
- ✅ Task 3 — Centipede hit spark effect
- ❌ .vscode/ — still correctly **not committed**
- 🚀 Next → **Task 4: Wave speed bonus**

Your latest push confirms:

⌵ Plain text

```
5257809..d0747fe main -> main
```

So we're clean. **Don't change anything else yet.**

### 🚀 Task 4 — Make later waves faster

Open `game.py` .

Find:

⌵ Python

```
def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the default
    pass
```

Replace **only that function** with:

⌵ Python

```
def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the default
    return 1 + 0.15 * (wave - 1)
```

### What this does

| Wave | Speed multiplier |
|------|------------------|
| 1    | 1.00×            |

| Wave | Speed multiplier |
|------|------------------|
| 2    | 1.15×            |
| 3    | 1.30×            |
| 4    | 1.45×            |
| 5    | 1.60×            |

So the game gradually gets harder. 🔥

## Then test

Run:

```
</> powershell  
  
py -3.12 game.py
```

Play for a bit and make sure the game still works normally.

**Don't commit yet.** First send me a screenshot or tell me that Task 4 is working, and I'll take you through the final commit + after-video step-by-step.

yes next'

Perfect bro 😎🔥 **Task 4 is the final coding task.** Now let's finish the assignment properly, step by step.

## ✅ STEP 1 — Save Task 4

Make sure `wave_speed_bonus()` is:

```
</> Python  
  
def wave_speed_bonus(wave):  
    """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the default  
    return 1 + 0.15 * (wave - 1)
```

Save `game.py` .

## ✅ STEP 2 — Test the game

Run:

```
</> powershell  
  
py -3.12 game.py
```

Check that:

- Game opens ☒
- Player moves ☒
- Shooting works ☒
- Mushrooms work ☒
- Spark appears when centipede is hit ☒
- Game doesn't crash ☒

If all good, close the game.

---

### ☒ STEP 3 — Commit Task 4

Run:

```
</> powershell  
  
git status
```

You should see something like:

```
</> Plain text  
  
modified: game.py  
untracked: .vscode/
```

**Do NOT add .vscode/ .**

Then:

```
</> powershell  
  
git add game.py
```

Then:

```
</> powershell
```

```
git commit -m "Task 4: Increase centipede speed by wave"
```

Then:

```
</> powershell
```

```
git push
```

Send me the output of `git push`.

## After that 🧑

We're basically done with the coding.

Then I'll guide you through:

- 5 Record AFTER video
- 6 Check GitHub
- 7 Create Lab-4 folder
- 8 Put before + after videos
- 9 Put updated code
- 10 Export ChatGPT chat history as PDF
- 1 1 Final submission checklist

And importantly, we'll make sure you have **separate commits for all 4 tasks**, as required by the lab instructions.  Lab\_4\_VibeCoding\_Student\_handout

```
PS C:\Desktop\SE LAB\27_centipede> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 341 bytes | 170.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local
objects.
To https://github.com/Suryapurimetla/27_centipede.git
d0747fe..a7f3683 main -> main
```

LETS GOOO BROOOO 🔥🔥 Task 4 is successfully pushed!



Your GitHub now has all 4 task commits:

1.  Task 1 — Fix mushroom HP
2.  Task 2 — Mushroom hit color
3.  Task 3 — Centipede hit spark
4.  Task 4 — Wave speed increase

Latest push:

 Plain text

```
d0747fe..a7f3683  main -> main
```

So coding part is DONE. 

## NEXT: Record the AFTER Video

The assignment asks for a **10-second video after making the changes.**

 Lab\_4\_VibeCoding\_Student\_handout

### 1. Start the game

Run:

 powershell

```
py -3.12 game.py
```

### 2. Record around 10–15 seconds

Show these things if possible:

-  Move the player
-  Shoot a mushroom
-  Show the mushroom changing color after being hit
-  Shoot a centipede segment
-  Show the spark effect
-  Continue playing briefly

You **don't need to demonstrate Wave 2** if it takes too long. The game running normally is enough for the video.

### 3. Stop recording

Save it as: